EECS 452 - Team 4 - Laser Tracking System

Sehyun (Sam) Park, Matthew Dubravec, Tyler Kegler, Ziyang (Jason) Ning, Christopher Bupp

April 24th, 2024

Table of Contents

I. Background & Motivations ---------- 1
II. Description ---------- 1
III. Important Technical Issues ---------- 1
IV. Testing ---------- 2

V. **Results** ---------- 6
VI. **Future Work** ---------- 7

Background & Motivations

The idea for our project originated from air defense systems where a flying target is identified and then destroyed. While most publicly acknowledged defense systems use material projectiles, there have been recent advancements in the field of directed energy weapons where LASER systems have allegedly been used to immobilize aerial vehicles, such as Russia's ZADIRA or Turkey's AKLA weapon systems. Given our shared interests in LASERs and as LASERs are effectively unlimited in range, we expected creating a line-of-sight LASER tracking system would lessen the complexity of the sensor system requirements as we could limit the system inputs to the 2D image plane. We then decided to use cameras for our primary sensors whose data we would process and send to two servos to adjust the position of a LASER in a x-y coordinate system created by the camera. Solutions similar to ours exist; however, most of the systems we found online used operating systems or were too slow for the kind of real-time motion tracking we wanted due to bloated libraries or inefficient coding languages. For the reasons mentioned above, we decided to design our code for each major aspect of our system and only use external libraries as needed.

Description

Our project focuses on integrating two primary hardware-software systems into one system. Beginning with object detection, we use an OV5640 camera sensor and run it on an ESP32-S3. The camera module communicates with the ESP via I2C and stores grayscale images in a memory buffer on the ESP. Our detection algorithm takes in the buffer values and, given a threshold brightness value, locates the median of pixel coordinates for pixels above the threshold. Since our camera is set up to output a 240x240 image, an 8-bit integer for each axis is sufficient to represent the x and y median coordinates, and our custom I2C wrapper sends this data to the Arduino. Our Arduino handles the reception of the ESP I2C transmissions and, once processed, transmits the necessary servo movement data to our two Dynamixel XL320 servos. As our camera system is directly mounted to our baseplate and remains stationary, we created an algorithm that maps the x and y coordinates from the camera to servo positions using a pixel-to-angle conversion. We then send the calculated positions to the servos which change the positioning of the LASER, thus keeping the LASER on target as the target moves.

Important Technical Issues

The implementation of cross-board communication came with several difficulties that were unexpected and poorly documented. The ESP32S3 Sense has SPI communication capabilities; however, the SD Card socket included with the Camera module occupied the SPI pins. We cannot conduct SPI communication without cutting the J3 solder pad on the PCB of the ESP32 [4], which is why SPI was not used in our implementation.

The I2C communication we utilized in the final iteration of the project brought about minor technical issues that are worth taking note of. The Arduino, when set to be an I2C slave, must be set up with a specific address unless there is the use of an I2C multiplexer. This address is 0x55

for the Arduino Uno. Another thing to note is that the Arduino and ESP32 Sense must have a common ground between the transmitter (the ESP32) and the receiver to read the data accurately. This can be accomplished by connecting both devices to the same source of power, or by connecting the two boards through the ground pins.

A grayscale image was chosen because of the increased memory space, memory access time, and computation required for an RGB565 image. RGB565 to RGB888 conversion is required for testing purposes since Pylot does not natively support RGB565 plotting. The conversion was tricky [1] [2] [3], requiring the team a week to implement properly both in Python and C. We discovered through testing that the raw RGB camera image was saturated, which made brighter colors appear white. With the discovery above, we chose a grayscale camera model to improve the speed of the detection algorithm.

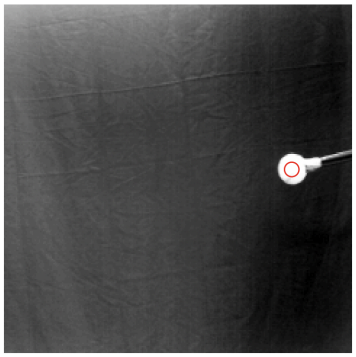
Figure 1: Image collected from OV5640 Camera using ESP32S3. The left shows a failed color conversion between RGB565 and RGB888, the right shows a tennis ball with saturated colors.

Testing

Our system has multiple systems that are set up independently of each other. We focused on unit testing each system component and integration testing with multiple components at the end.

Camera and Detection Algorithm Testing: Camera data must be verified to ensure accurate output and correct image size. To test the camera, we iterated through the frame buffer and printed each pixel value to the serial monitor. This text data was copied to a CSV file which the Python testing script used as an input to plot the image.

We wrote the detection algorithm first in Python as a proof of concept. The CSV files from the camera tests were used as inputs, and we wrote functions to emulate the ESP32's frame buffer. The Python prototype algorithm did not use any Python-specific libraries to ensure a seamless translation to the C programming language. The detection output is then plotted on top of the image to validate the accuracy, for example, provided in **Figure 2.** After testing the Python detection algorithm, we translated the code into C with the Arduino IDE. Aside from ensuring the detection works as intended, we completed extensive profiling to check for timing constraints, which will be discussed in the **results** section.



```
___________ONE PICTURE__________
135374
frame loader time:, 128152
Median algo time:, 7118
___________ONE PICTURE__________
135422
frame loader time:, 128149
Median algo time:, 7112
___________ONE PICTURE__________
135414
```

```
___________ONE PICTURE__________
50606
frame loader time:, 43866
Median algo time:, 6612
___________ONE PICTURE__________
50628
frame loader time:, 43838
Median algo time:, 6614
___________ONE PICTURE__________
50602
```

Figure 2: The left shows an output image of the python implementation of the detection algorithm. The middle and the right shows the profiling result of the C code, units in microseconds.

Servo System Identification:

Having a knowledge of the physical properties of the servos are crucial for achieving high controllability and understanding of system delays. To achieve this, we had to apply system identification to create mathematical models for further analysis. The system identification was done using the MATLAB's system identification toolbox where it returns estimated transfer function of the system/plant when input and output signals are given as input parameters. This was achieved using the Dynamixel Wizard where it possessed graphing of input and output servo positions, similar to Figure 3. We measured the servo positions in several frequencies by creating the chirp signal. Measurement of high frequencies were unnecessary as our actual implementation would not require such instantaneous movements. The data was run using estimation of 2 poles and 1 zeros which resulted in a transfer function with 92.9% estimation accuracy.

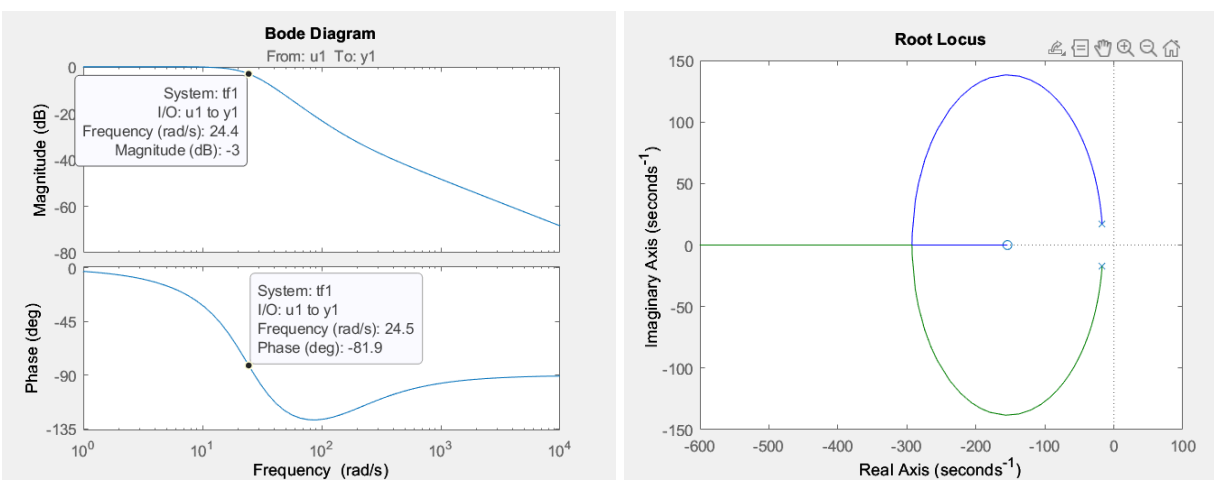


Figure 3: Plot on the left shows the bode diagram and right shows the discrete root locus plot from the resulting transfer function.

Using this transfer function, we were able to do further analysis by creating a bode plot and root locus plot. The bode plot shows the bandwidth frequency where the output signal's amplitude drops to -3 dB compared to the input signal. It signifies the limit for tuning our system, beyond which the motor's dynamics cannot be effectively adjusted. Any input signals beyond this frequency will be significantly reduced in magnitude. The root locus plot shows the

stability of our current motor system where all the poles lie on or inside the unit circle, which signifies the stability of our system.

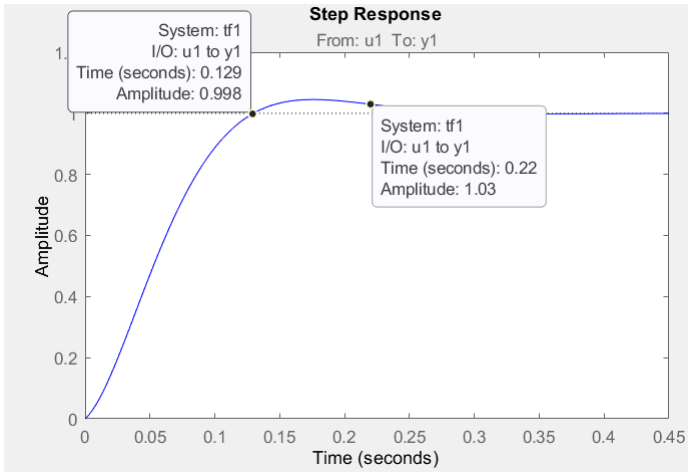


Figure 4: The step response of the resulting transfer function.

Figure 4 shows the step response of the system. We can determine the rise time and settling time of the estimated mathematical model of the system by inspection. The rise time of this model concluded as 0.129 seconds with 3% settling time of 0.22 seconds. 0.22 seconds can be considered to be slow but this data was measured using ±30 degrees, our edge case for the limited boundary condition for our servos. We concluded that additional PID tuning was needed to enhance the responsiveness of our servo system.

Servo Testing:
Initial servo testing was completed using the default servo PID gain settings. The servos were commanded to light their LED and move to ±3 degrees. We measured the response time of the servo by taking a slow-mo video using a smartphone camera and counting the frames from the moment the LED light lit up to the servo moving away from the initial position. The average response time for this experiment was 20ms with a 7.5V power source, so we decided additional PID gain tuning was required.

PID tuning was completed using Dynamixel Wizard 2.0, a powerful visualization tool provided by the XL320 motors. The Dynamixel wizard allowed us to do real-time PID tuning as it would graph a chart of the input that we sent the motor versus the output that the motor was physically doing. The Dynamixel wizard would also allow us to change these PID values in real time, seeing the effects of increasing the P, increasing the I, and increasing the D. **Figure 5** shows the graphs of the motor's collected response in real time. The graph to the left is the PID default mode, which you can see has overshoots and issues with settling time. This results in the motor overcorrecting and needing to come back, but it cannot do so because the PID values are too small, and the default value for the PID is a P of 32. The graph on the right shows a completed tune of the PID for our motors and great response with low settling time. This was done by having a P of 36, I of 3, and D of 16 and an iteration process of adjusting values inside the PID until the motor response was acceptable for the application. This iteration took many times before we concluded that these PID values were adequate for the response of our system. Our current setup goes from our maximum set point to our minimum (±30 degrees) in about

300 milliseconds. PID tuning often involves balancing between achieving a fast response, minimal overshoot, and maintaining system stability. This iteration process requires careful observation, analysis, and adjustments to achieve the desired behavior of our motors.

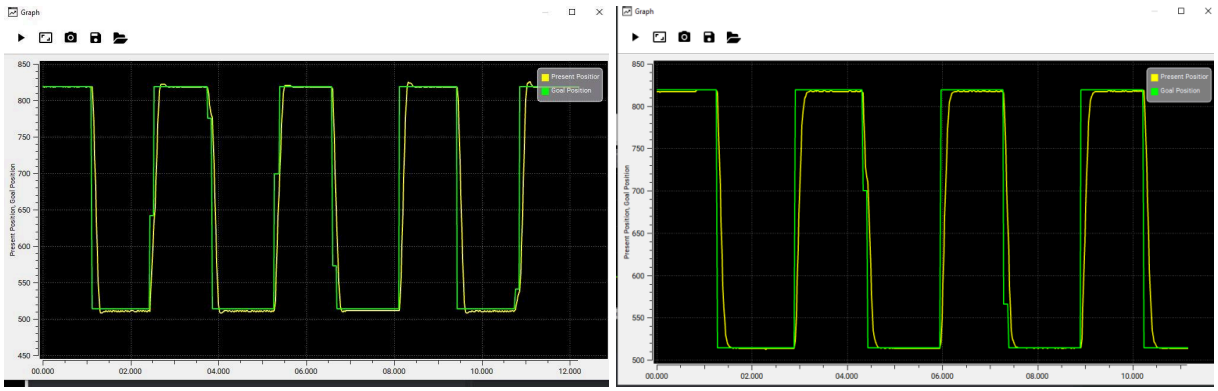
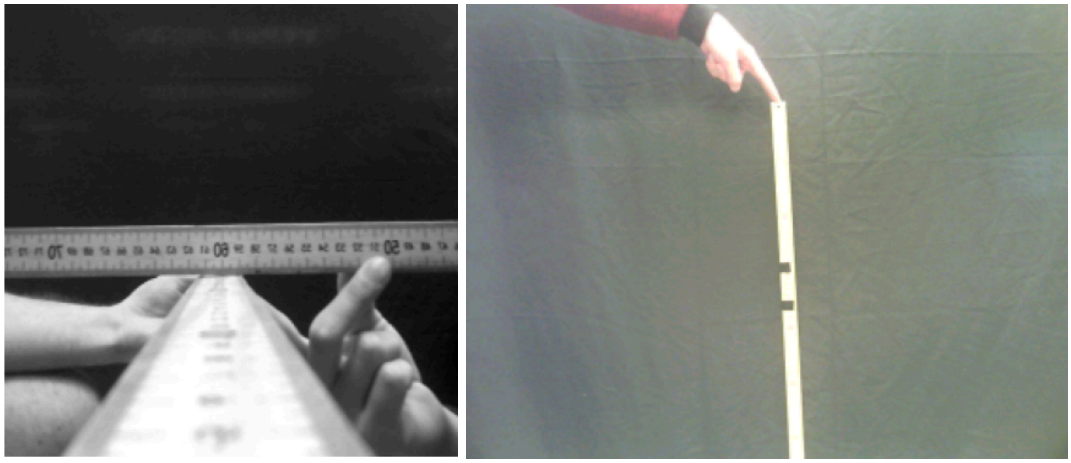


Figure 5: The graph to the left is the original PID response and the graph to the right is the tuned PID response.

Servo XY Mapping Testing:
We implemented a custom servo movement algorithm to update the servos to positions corresponding with the median x and y coordinates. The primary idea behind our movement algorithm was to map the 240x240 image that the camera captured to our servos' bitwise positioning. We started by finding the maximum angle of view from our OV5640 by taking a picture of a meter stick from a set distance with a 5 cm gap marked on the stick. We then counted the number of pixels in the 5 cm gap and used those values to calculate the camera's angle of view using simple geometry. This value was approximately 48 degrees by 48 degrees for the 240x240 image.



Figures 6: Images used to calculate the x and y angle of views for OV5640.

We then found the minimum movement angle that the servos were capable of (approximately .29 degrees) and found the total number of bits required to move the servos across the x and y axes of the camera frame. This maximum bit movement value was then divided by 240 to give the approximate bit-to-pixel value that, once combined with an offset, allowed us to directly map the target position on the backdrop (sent as median values from the ESP) to a cartesian 2D plane. To make the servos correctly update their positions to the median values sent from the

ESP32, we created an interrupt system where, once another median position was sent to the Arduino, the system would interrupt the current signal and send the updated position requirement to the servos rather than having the servos complete their movement before accepting the next position. This allowed us to have better results as the response time increased and the moment of inertia of the servo gears was conserved in most cases (saving as much as 20-40 ms depending on the servo movement requirement).

Communication Testing:
Communication was tested using a simple set of programs that sent information to the Arduino from the ESP32 in two-byte sets before printing the data sent to the serial monitor attached to the Arduino. The sent information consisted of the coordinates (100, 100) and (0, 0) being sent every 500 ms to the Arduino. The Arduino then printed the received coordinates in response to two bytes of information being sent. More testing was completed after establishing basic communication.

With a basic connection established, communication testing was made more rigorous by sending actual calculated positions to the Arduino based on the movement of a Tennis Ball. The ESP32 Sense would print the calculated ball position before sending the position to the Arduino. The Arduino would print out the received information, which was verified to be correct and done quickly. The process of sending information from the ESP32 to the Arduino took 360 μs on average.

```
sending data time:, 356
92, 150
sending data time:, 360
92, 150
sending data time:, 356
92, 150
sending data time:, 359
92, 150
```

Figure 7: Profiling result of sending position information from the ESP32 Camera code sent to the Arduino, time unit is in microseconds.

Mount of Laser Targeting System Testing:
From the beginning of this project, we knew that we needed custom mounts for the motor set we used. Once the team chose to use the XL 320s from Dynamixel, we began making motor mounts. The final mount setup was a two-axis system with the x-axis motor mounted to a base and the y-axis motor mounted to the x-axis wheel. This gave our design a full range of movement and the horizontal and vertical planes. The x-axis motor was mounted to a base, providing stability as the motors quickly updated their positionings. The y-axis motor was attached to the x-axis motor using a 3D-printed mount from Dynamixel. This amount is the fpx330 - h101 and is available in Step format on the Dynamixel website. The motor was attached using zip ties that went through the motor mounts on the Dynamixel XL 320.

The next step in this design process was to make a mount to secure the baseplate to the x-axis motor. The first motor base design broke in the 3D printer, and the holes (intended for zip ties) were too small, forcing us to scrap it. The next motor base (Motor base 2.0) used a set of holes that were in line with the shelves of the motors and would allow the motor to sit, making for a

secure lock. Added holes in the bottom of the bracket allowed zip ties to create a secure connection with the base plate. This then allowed the motors to move at maximum speed without play.

The last component designed for our system was the y-axis motor mount that would hold the laser and the ESP32 with its camera. This design needed the horizontal and vertical axis in the middle where the laser is. We edited fpx330-h101 by removing most of the material and only keeping the side panel for mounting purposes. We then decided to make a separate camera mount for the ESP32 and the camera. The stationary mount connected to our base plate and pointed directly at our background. We later updated this design (now Camera Mount 3.1) when we decided to extend the plates that held the ESP32, allowing us to have a stronger hold of our ESP 32 and making better housing and protection of our system. With the design change of having a stationary camera, we also needed to redesign the laser mount. We found dimensions that worked well for the laser and removed unnecessary features, just allowing a single hole to hold the laser. This is known as laser Mount 1v. The final design of our system integrated Laser mount 1v, Camera mount 3.1, Motor base 2.0, and fpx330 - h101 to make our system work.

Results

We are pleased to report that our final setup met our project expectations across all categories. Our results show that we were processing and sending 19-20 frames per second from the ESP to the Arduino. Likewise, our servo response and accuracy times met our expectations with our servos following a randomly moving object with an average speed of 0.3 m/s and a 72% time on target accuracy.

Final Image Processing Results:
Our final image processing setup resulted with a transmission frame rate averaging approximately 19-20 frames per second. This frame rate includes the OV5640 capturing an image, accessing the image in memory, executing a thresholding and running median algorithm on the stored image, and transmitting the median x-y coordinates to the Arduino via I2C. Due to this high framerate and the robustness of our algorithms, we successfully mitigated the effects of motion blur on our image processing system, even when moving our target at 2-3 times our expected target movement speeds (0.3 m/s).

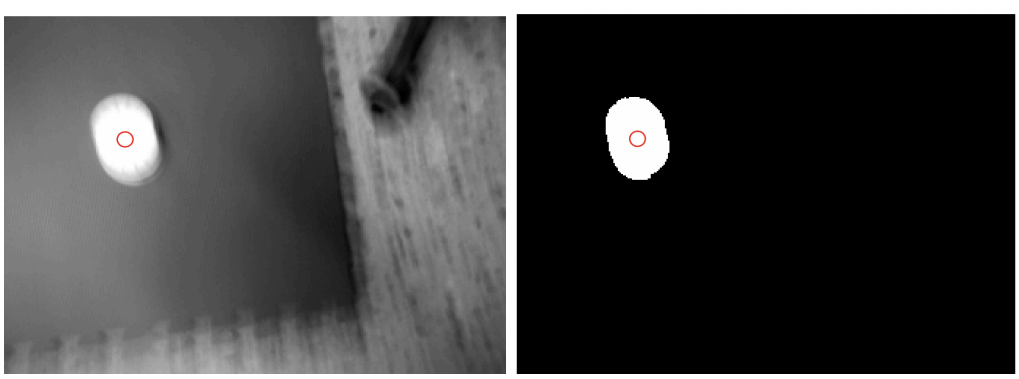


Figure 8: Input image with motion blur and non-uniform background, output of object detection with red dot denoting the predicted location

	Median + Thresholding Algorithm (μs)	Frame Loader (μs)	Total Runtime (μs)	Percent Time Spent Grabbing Image	Average FPS
Image Size (320X240)	7112	128149	135414	94.6%	7.4
Image Size (240X240)	6585	43874	50610	86.7%	19.7

Figure 9: Results from testing showcasing the timing of the detection algorithm, the average FPS, and the percentage of time spent on grabbing image data from the frame buffer.

Final Servo Results:
Our final servo setup, when integrated with the ESP outputs and I2C communications, resulted with approximately 72% time on target accuracy with a target moving at 0.3 m/s. The results are shown in **Figure 10** where each column indicates the relative movement of the target, and the cell values show the average speed of the target for the one video data and the time on target for that video.

Down Movement (m/s, %)	Up Movement (m/s, %)	Left to Right Movement (m/s, %)	Right to Left Movement (m/s, %)
0.16, 100	0.16, 100	0.46, 40	0.48, 100
0.16, 99	0.37, 98	0.68, 9.7	0.37, 72
0.35, 76	0.41, 90	0.5, 19.5	0.452, 76
0.35, 77	0.27, 99	0.38, 27	0.453, 68
0.26, 63	0.48, 72	0.14, 75	0.33, 83
0.17, 91	0.425, 87.5	0.24, 88	0.4, 22
0.2, 77		0.24, 84	
		0.08, 96	
		0.46, 39	

Figure 10: Results from taking videos of the system tracking a moving target and finding the average speed of the target in the video (m/s) and the corresponding time on target accuracy

Future Work

Our demo showed that our system is offsetted by some degree in our servo movement. The detection algorithm for detecting the center of our object is found to be consistent and accurate. This means the pixel-to-servo angle translation has to be improved or applied with an alternate algorithm for better servo movement. One option to improve our pixel-to-servo angle translation would be to use better, brushless servos. Though significantly more expensive than the XL320s, brushless servos would allow for faster and more consistent servo adjustments. After accounting for accurate servo movement, we could add an estimation algorithm to compensate for the servo response time and internal system delay. One of the possible solutions for this is the Smith prediction algorithm, which uses a process model to predict future values of the output variable. Additional solutions that would require more computational power are advanced Kalman Filters or even machine learning models. Both of these algorithms would likely increase the time-on-target accuracy of our system if integrated with a stronger processing chip. Lastly, if we were to have more processing power by using a more expensive processing chip, we would likely be able to optimize our response times by increasing the frames per second from our camera.

Our current design does not have a proper mount for the boards and wires unorganized. Our team could improve our physical packaging to make our system more presentable.

Citations:

[1] Lee, Josh, et al. "How Does One Convert 16-Bit RGB565 to 24-Bit RGB888?" *Stack Overflow*, 1 Dec. 1955, stackoverflow.com/questions/2442576/how-does-one-convert-16-bit-rgb565-to-24-bit-rgb888.

[2] Unknown, Unknown. "RGB565 Color Picker." *RGB Color Picker*, 2024, rgbcolorpicker.com/565.

[3] Xenon, Xenon, and ESP_Me-no-dev Unknown. "Espressif Forum." *ESP32 Cam Direct Access to Image Data - ESP32 Forum*, 2019, www.esp32.com/viewtopic.php?t=10405.

[4] *Pin multiplexing with seeed studio Xiao ESP32S3 (sense): Seeed Studio Wiki*. Seeed Studio Wiki RSS. (2023, March 30). https://wiki.seeedstudio.com/xiao_esp32s3_pin_multiplexing/

Appendix:

Within the angle_translation.h, there is a fast atan formula used in the fast_atan() function. This originates from the following website: https://nghiaho.com/?p=997, which credits the formula to a paper called "Efficient approximation for the arctangent function", Rajan, S. Sichun Wank Inkol, R. Joyal, A., May 2006. This was used over the atan function within the math library for Arduino.